OS - Unit 4 - Orange Problem

Name : Nishkaa V
SRN : PES2UG24CS907
Section : 4B
Github link : https://github.com/Nishkaa25/PES2UG24CS907-pes-vcs

Screenshots:

Phase 1 :

1A

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$
```

1B

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$
```

Phase 2:

2A

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$
```

2B

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ find .pes/objects -type f
.pes/objects/f8/b93f79591de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ xxd .pes/objects/f8/b93f7959
1de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9 |head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066  tree 48.100644 f
00000010: 696c 652e 7478 7400 aaaa aaaa aaaa aaaa  ile.txt.........
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa  ................
00000030: aaaa aaaa aaaa aaaa                      ........
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$
```

Phase 3

3A

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ echo "hello" > file1.txt
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ echo "world" > file2.txt
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./pes add file1.txt file2.txt
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  index.c
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  make_tree.c
  untracked:  test_tree
  untracked:  object.c
  untracked:  make_tree
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  README.md
  untracked:  tree.c
  untracked:  tree.h
  untracked:  index.h
```

3B

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776452072 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776452082 6 file2.txt
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$
```

Phase 4

4A

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ ./pes log
commit 400583644a9ddfd09075ebe250c06982afa83019db369605a1174ca1a39f752e
Author: Nishkaa PES2UG24CS907
Date:   1776453014

    Add farewell

commit b009c00d3a1f271f9224366b06d0bd91b7d5bdb079735bdf4c3c456287d98b47
Author: Nishkaa PES2UG24CS907
Date:   1776452989

    Add world

commit 07c80b9ab3ee0e7d42e3aff8bfd6d590a5c2171a762d0949fb01e5ebba1e67fa
Author: Nishkaa PES2UG24CS907
Date:   1776452958

    Initial58

Inia
```

4B

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/07/c80b9ab3ee0e7d42e3aff8bfd6d590a5c2171a762d0949fb01e5ebba1e67fa
.pes/objects/40/0583644a9ddfd09075ebe250c06982afa83019db369605a1174ca1a39f752e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b0/09c00d3a1f271f9224366b06d0bd91b7d5bdb079735bdf4c3c456287d98b47
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C

```
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
nishkaa@nishkaa-VirtualBox:~/PES2UG24CS907-pes-vcs$ cat .pes/refs/heads/main
400583644a9ddfd09075ebe250c06982afa83019db369605a1174ca1a39f752e
```

Phase 5

1. How would you implement pes checkout <branch>?

To implement pes checkout <branch>, the system must first check whether the given branch exists in .pes/refs/heads/. If the branch is found, the HEAD file should be updated to point to that branch reference.
Next, the commit hash stored in that branch file is read. Using this commit, the corresponding tree object is obtained, which represents the snapshot of the project. The working directory must then be updated to match this snapshot by creating required files, updating modified ones, and removing files that are not part of the target commit.
Before performing this operation, it is important to ensure that there are no uncommitted changes in the working directory, otherwise data loss may occur.

2. How to detect a dirty working directory?

A working directory is considered dirty when its current state differs from the state recorded in the index.
This can be detected by checking each indexed file against the filesystem using metadata such as file size and modification time. If any of these values differ, the file has been modified. If a file listed in the index is missing, it has been deleted. Additionally, any new files that are not present in the index are considered untracked.
For more reliable detection, the file's content hash can also be recalculated and compared with the stored hash in the index.

3. What is a Detached HEAD?

A detached HEAD state occurs when the HEAD does not point to a branch but instead directly references a specific commit.
In this situation, any new commits created are not associated with a branch. As a result, if the user switches to another branch, these commits may become unreachable and eventually lost unless their commit hash is explicitly saved.
To retain such commits, a new branch can be created pointing to the detached commit before moving away.

Phase 6

1. Garbage Collection Algorithm

Garbage collection is used to remove objects that are no longer reachable from any branch.
The process begins by identifying all branch heads and marking their associated commit objects as reachable. From each commit, its parent commits and tree objects are explored. The tree objects are then traversed to identify all referenced blobs and subtrees.
A traversal technique such as depth-first search or breadth-first search can be used while maintaining a set of visited object hashes. After marking all reachable objects, any object not present in this set is considered unreachable and can be safely deleted.

2. Why is GC dangerous during a commit?

Garbage collection can interfere with an ongoing commit because newly created objects may not yet be fully linked through references.
If garbage collection runs during this time, it may incorrectly identify these newly created objects as unused and delete them. This can lead to incomplete or corrupted commits.
To avoid this issue, systems ensure that object creation and reference updates are done atomically, and garbage collection is either delayed or protected using locking mechanisms.